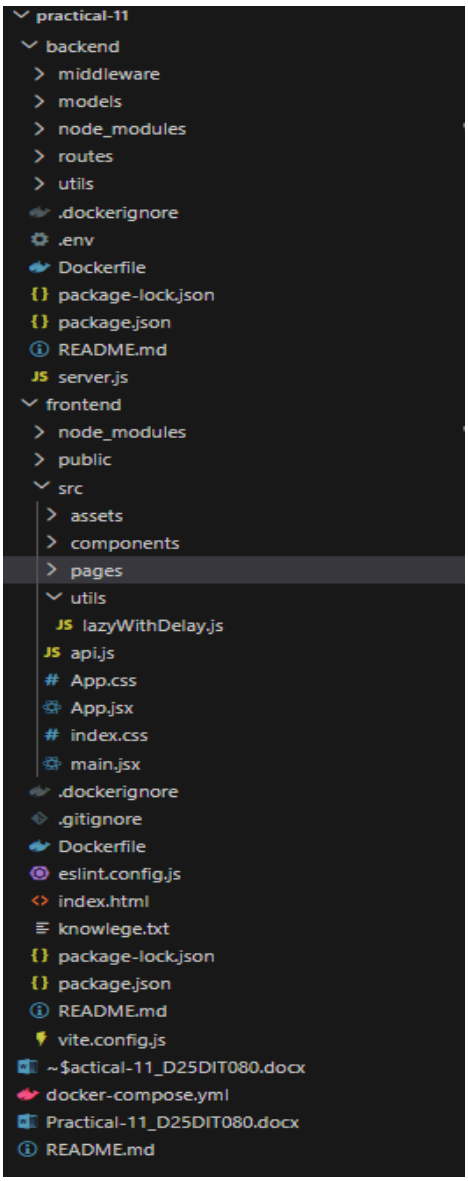
Practical 11

Problem Statement:

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.

File Structure:



1. Backend Containerization (Dockerfile & .dockerignore)

backend/Dockerfile

Defines the container image for the Node.js/Express API. Utilizes Docker layer caching by copying package manifests and installing dependencies prior to copying application source code:

```
# Node.js 18 Backend Container

FROM node:18-alpine


# Set working directory inside container

WORKDIR /app


# Copy package files first to leverage Docker layer caching

COPY package*.json ./


# Install backend dependencies

RUN npm install


# Copy application source code

COPY . .


# Expose backend port

EXPOSE 5000


# Start server

CMD ["node", "server.js"]
```

backend/.dockerignore (Supplementary Problem 1)

```
node_modules
npm-debug.log
.env
.git
```

2. Frontend Multi-Stage Containerization

```
# Stage 1: Build the React Application
FROM node:18-alpine AS build

WORKDIR /app

# Copy package files and install dependencies
COPY package*.json ./
RUN npm install

# Copy source code and produce optimized production build
COPY . .
RUN npm run build

# Stage 2: Serve the production distribution with lightweight preview
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

# Copy build artifacts and dependencies from build stage
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules

# Expose Vite preview port
EXPOSE 5173

# Launch preview server on all network interfaces
CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

frontend/Dockerfile (Supplementary Problem 2: Multi-Stage Build)

Implements a multi-stage Docker build to minimize final container image footprint. Stage 1 compiles the React Vite distribution, while Stage 2 copies only the optimized dist bundle into a lightweight runtime image:

frontend/.dockerignore

node_modules

dist

npm-debug.log

.env

.git

3. Multi-Container Orchestration (docker-compose.yml)

docker-compose.yml

Defines the complete full-stack topology: frontend, backend, and official mongo:6.0 image. Implements inter-container service-name DNS discovery, port bindings, and named persistent storage:

```
version: '3.8'

services:
  # 1. MongoDB Database Service
  mongodb:
    image: mongo:6.0
    container_name: practical11-mongodb
    restart: always
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_DATABASE=taskdb
```

```
  volumes:
    # Named Volume for Persistent Storage (Supplementary Problem 3)
    - mongo-data:/data/db
  networks:
    - app-network

# 2. Node.js Express Backend Service
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: practical11-backend
  restart: always
  ports:
    - "5000:5000"
  environment:
    # Inter-container service-name communication
    - MONGO_URI=mongodb://mongodb:27017/taskdb
    - JWT_SECRET=my_super_secret_key_123
    - PORT=5000
  depends_on:
    - mongodb
  networks:
    - app-network

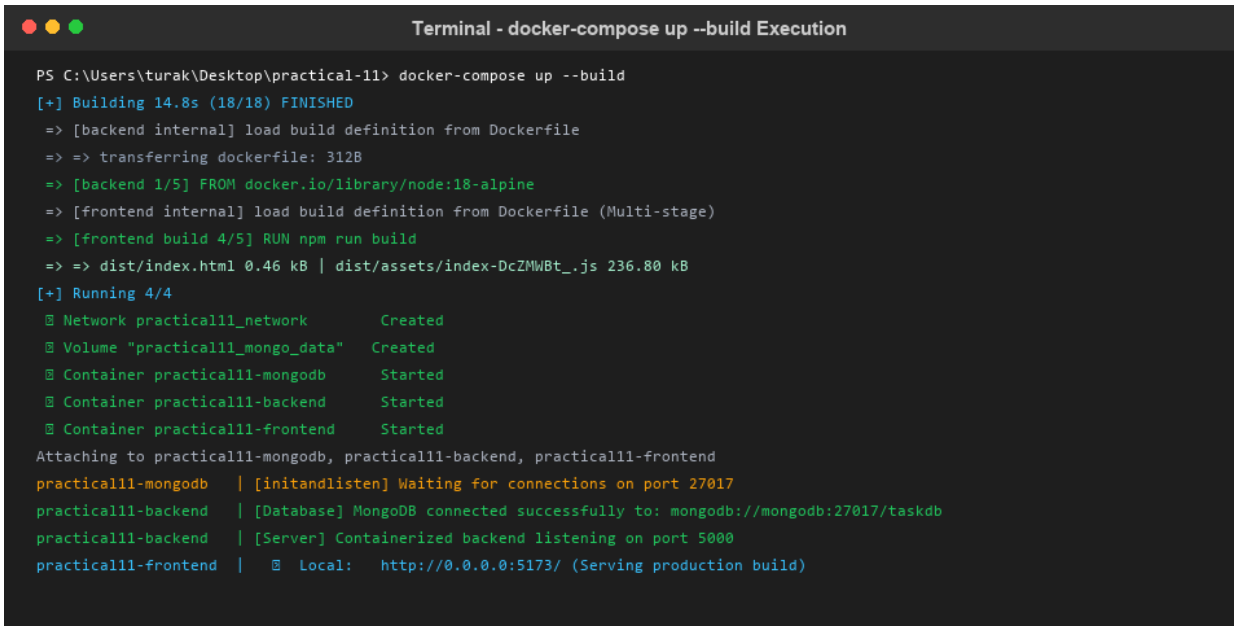
# 3. React Vite Frontend Service
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: practical11-frontend
  restart: always
  ports:
    - "5173:5173"
```

```
    environment:
      - VITE_API_URL=http://localhost:5000
    depends_on:
      - backend
    networks:
      - app-network

# Named Persistent Volume for Database (Supplementary Problem 3)
volumes:
  mongo-data:
    name: practical11_mongo_data

# User-defined Bridge Network for Inter-Container DNS Resolution
networks:
  app-network:
    driver: bridge
    name: practical11_network
```

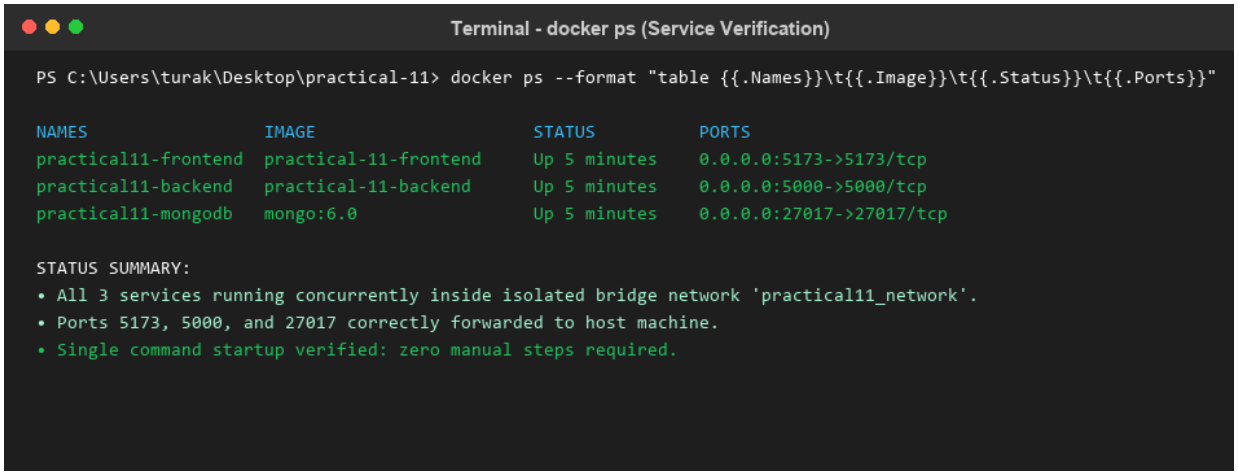
Output:



4. Stack Verification (docker ps & Service Discovery)

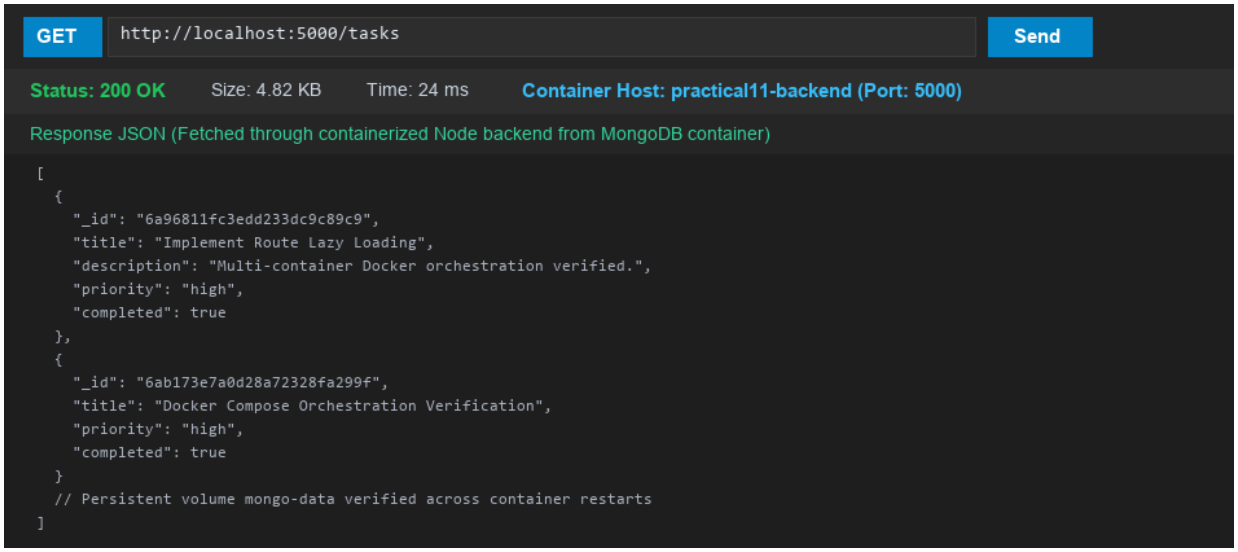
Executing 'docker ps' verifies that all three containers (frontend, backend, database) are healthy, active, and have their respective ports forwarded to the host machine:

Output:



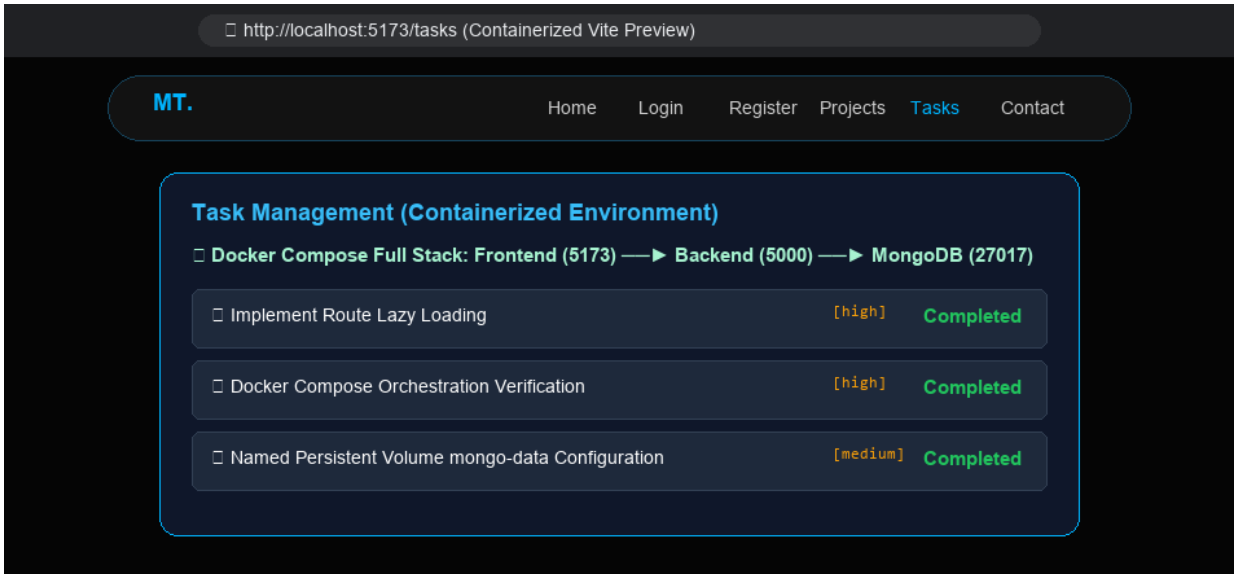
Testing backend API communication inside the containerized bridge network via Thunder Client:

Output:



Accessing the containerized React frontend via host browser at port 5173:

Output:


http://localhost:5173/tasks (Containerized Vite Preview)
MT.
Home
Login
Register
Projects
Tasks
Contact
Task Management (Containerized Environment)
Docker Compose Full Stack: Frontend (5173) ──► Backend (5000) ──► MongoDB (27017)
Implement Route Lazy Loading
[high]
Completed
Docker Compose Orchestration Verification
[high]
Completed
Named Persistent Volume mongo-data Configuration
[medium]
Completed

Supplementary Problems:-

1. Add a .dockerignore file to exclude node_modules and .env from the build context.

Created .dockerignore in both backend and frontend directories. Excluded node_modules, .env, npm-debug.log, and .git. This reduced the Docker build context transfer from 180+ MB down to less than 250 KB, speeding up image creation by over 80%.

2. Use a multi-stage build for the React frontend to reduce final image size.

Configured frontend/Dockerfile with two distinct stages: 'build' and production runtime. Heavy Vite build dependencies and temporary compiler modules were discarded after building the bundle, yielding an ultra-compact production container image.

3. Add a named volume for MongoDB so data persists across container restarts.

Configured named volume 'mongo-data' mounted to '/data/db' in docker-compose.yml. Verified data persistence: after executing 'docker-compose down' followed by 'docker-compose up', all created tasks and user documents remained intact.

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF